

QuakeWatch: Design Document

Oct 3, 2026 · @Karthik Mohan · v0.1.0

This document describes how QuakeWatch v0.1.0 is built: its components, data flow, data model, the algorithms that keep loads correct, its interfaces, how it fails and recovers, and the decisions behind it. Goals, plan, and measured results are in the project document.

System in one line: local Python commands fetch bounded USGS earthquake queries into audited JSONL batches, load them into Snowflake RAW with `COPY INTO`, and a Snowpark procedure turns each loaded batch into revision-aware facts in one transaction. Source code: github.com/kaarthikmohaana/quakewatch.

Goals and non-goals

Goals

- Every loaded window is auditable: requested parameters and before/after source counts are recorded, and RAW rows reconcile to them.
- Loads are idempotent: rerunning or overlapping a batch converges to the same facts.
- Revisions and deletions are kept as history, so the warehouse can answer what changed and when.
- Any failed transform can be retried from RAW without refetching the source.
- Every command that touches USGS or Snowflake previews by default and acts only when asked.

Non-goals

- Streaming or sub-minute freshness; there is no Snowpipe Streaming, Kafka, or Snowflake Stream.
- A scheduler or orchestrator; runs are started by hand in v0.1.0.
- Proof of complete coverage; USGS offers no consistent catalog snapshot, so gaps are reported rather than ruled out.
- Hazard estimates of any kind: distance and magnitude are descriptive only.

Design principles

Principle	How it shows up
Bounded and audited	Every request has fixed time bounds, a count before and after, and a manifest. A window that does not reconcile becomes an explicit coverage gap, never a silent omission
Append, never overwrite	Receipts, processing attempts, and revisions are appended. A retry gets a new ID, so earlier failures stay on record

Principle	How it shows up
Reconcile before trusting	HTTP 200 is never proof. Counts are compared at fetch, at <code>COPY</code> , in RAW, in staging, and in the health view
One writer per table	Python and <code>COPY</code> own RAW; the Snowpark procedure owns everything curated; views own derived state
One transaction per batch	All model writes for a batch commit together with its success record, or roll back together
Preview by default	Commands print what they would do; <code>--execute</code> (or <code>EXECUTE=1</code>) is required to contact USGS or Snowflake
Public data only	Example sites use public city-center coordinates; credentials and private coordinates never enter Git

Architecture

Two runtimes: a local Python 3.12 command line that talks to USGS and Snowflake, and a Python 3.12 Snowpark procedure that runs inside Snowflake. Snowpark is never imported locally.

```

USGS FDSN event API (count + query endpoints)
| bounded requests, count before and after
v
Local extractor (history or update sweep)
-> data/raw/<attempt-id>/events.jsonl + manifest.json
v
RAW loader: PUT to @USGS_JSON_STAGE/<attempt-id>/ -> COPY INTO
-> RAW.RAW_EVENT_RECORDS (payload VARIANT) + RAW.BATCH_ATTEMPT receipt
v
CALL CURATED.PROCESS_LOADED_ATTEMPT(attempt_id) [one transaction]
-> STG_EVENT_REVISION -> FACT_EVENT_REVISION, BRIDGE_EVENT_SITE,
    dimensions, FACT_BATCH_RUN, BATCH_PROCESS_ATTEMPT
v
Views: EVENT_CURRENT, V_BATCH_HEALTH, V_RECEIPT_WINDOW_AUDIT, V_EVENT_REJECTS
v
Read-only checks and analysis SQL; Cortex brief checked against SQL facts

```

Components

Component	Runs in	Responsibility	Code
History extractor	Local	Plans 180 month windows and fetches them with count checks, splitting, retries, and checkpoints	<code>extract_batch.py</code> , <code>history_plan.py</code>

Component	Runs in	Responsibility	Code
Update-sweep extractor	Local	Fetches everything updated after the watermark minus an overlap, catalog-wide	update_plan.py, update_extract.py
RAW loader	Local → Snowflake	Validates a complete batch, stages it, runs <code>COPY INTO</code> , reconciles counts, appends a receipt	raw_load.py, history_raw_load.py
Update loader	Local → Snowflake	Loads a completed sweep and advances the watermark by compare-and-set	update_load.py
Processing procedure	Snowflake (Snowpark)	Projects, deduplicates, resolves aliases, and merges one attempt into the model	snowpark_*.py, process_*.py, revisions.py, aliases.py, staging.py
Views	Snowflake	Current state, batch health, window audit, rejects	sql/setup/07, 10
Checks	Local → Snowflake	Reconciliation, uniqueness, latency, analysis, and the live integration check; all read-only	scripts/checks/
Cortex brief	Snowflake Cortex	Summarises a SQL aggregate; every fact is screened against the SQL	scripts/evidence/phase4/

Snowflake objects

Object	Setup
Database and schemas	<code>QUAKEWATCH</code> with <code>RAW</code> (landing, receipts, watermark) and <code>CURATED</code> (model, views, procedure)
Warehouse	<code>QUAKEWATCH_WH</code> : X-Small, auto-suspend 60 s, auto-resume, created suspended
Stage	<code>RAW.USGS.JSON_STAGE</code> , internal, JSON file format; one directory per attempt
Role	<code>QUAKEWATCH_ROLE</code> , granted create table/view/procedure on the two schemas; the project key is restricted to it

Object	Setup
Procedure	<code>CURATED.PROCESS_LOADED_ATTEMPT(ATTEMPT_ID)</code> , Python 3.12 and Snowpark 1.55.0, imported from a ZIP bundle on the stage
Setup order	<code>sql/setup/01-10:01</code> once as admin, then <code>make bootstrap EXECUTE=1</code>

Extraction

Request contract

Parameter	Value
Endpoints	<code>/fdsnws/event/1/count</code> and <code>/fdsnws/event/1/query</code>
Format	<code>geojson, orderby=time-asc, limit=10000</code>
Time bounds	<code>starttime</code> and <code>endtime</code> in UTC; history uses origin time
Location	<code>latitude, longitude, maxradiuskm=250</code> for the site (history only)
Deletions	<code>includedeleted=true</code> , so deleted events arrive as tombstones
Client	HTTPX, 30 s request timeout, identifying User-Agent

Window protocol

1. Plan 60 contiguous month-sized windows per site from the fixed cutoff (180 in total). Month windows replaced year windows after a year-long count request timed out.
2. Ask the count endpoint for the window. At 10,000 or more, split before fetching; USGS caps a response at 20,000.
3. Fetch the features, then count again. The window reconciles only when count before, rows returned, and count after all agree; otherwise retry, then split.
4. Split at the midpoint. Both halves include the midpoint, so the boundary cannot be missed; each row keeps its window ID and duplicates are removed downstream.
5. Record every leaf and parent in the manifest's window audit. Only a fully reconciled attempt writes `events.jsonl`.

Limits and retries

Limit	Value
HTTP attempts per request	4, for transport errors and statuses 429, 500, 502, 503, 504
Consecutive read timeouts	2, then the request fails
Backoff	$2^{(n-1)}$ seconds capped at 8, plus up to 0.25 s jitter; <code>Retry-After</code> honoured up to 30 s
Reconciliation attempts per window	3
Timeout split depth	3 levels
Window budget	180 s wall clock, shared by all retries and splits of a source window
Slow months	Optional child slices of 7 days, 1 day, or 3 hours, each reconciled on its own

When the budget expires, or the command is interrupted or terminated, the attempt is marked `failed` with zero written rows and an explicit coverage gap. The manifest records the active window and saves each completed child before the next starts, so a failure shows exactly where it stopped.

Checkpoints: with `--resume-children`, each reconciled child is saved under `children/` and reused only when its logical batch, slice size, query parameters, content hash, and counts all match. Reused rows keep their original fetch time.

Output envelope

Field	Meaning
<code>source_feature</code>	The full USGS GeoJSON feature, unmodified
<code>metadata.logical_batch_id</code>	Stable ID for the requested site and time range; groups retries
<code>metadata.attempt_id</code>	Unique ID for this execution
<code>metadata.window_id</code>	Query window that returned the row; split children get child IDs
<code>metadata.fetched_at</code>	UTC capture time
<code>metadata.payload_hash</code>	SHA-256 of the canonicalised feature JSON
<code>metadata.parser_version</code>	Version of the raw envelope

Update sweep

The history load finds events by origin time; the sweep finds changes by update time, so revisions and deletions to old events are captured even when an event moves across a site boundary.

Aspect	Design
Inputs	Catalog origin-time lower bound, fixed cutoff, prior committed watermark, sweep start time, positive overlap in seconds
Query	<code>updatedafter = prior watermark - overlap; starttime = catalog lower bound; endtime = cutoff; no spatial or magnitude filters; deletions included</code>
Slicing	Origin-time slices sized by counts: 50 years by default, smaller for recent years (for example 5-year from 2001, monthly from 2023, daily for one slow month)
Deadline	One 3-minute deadline for the whole attempt
Output	A unique manifest with <code>batch_kind: update_sweep</code> ; history loaders ignore these attempts
Watermark	Extraction never advances it; the manifest keeps <code>watermark_advanced: false</code>

Load and commit

1. Validate the completed sweep locally: contiguous coverage, every leaf's counts, and rows per window.
2. Read the prior committed watermark from `RAW.UPDATE_WATERMARK` and reject a stale sweep, a changed lower bound, or a backward cutoff.
3. Load the attempt into RAW, or recognise an earlier load of the same attempt.
4. Compare the stored receipt manifest and per-window RAW counts with the local sweep.
5. In a transaction, advance the watermark to the sweep start with `UPDATE ... WHERE COMMITTED_WATERMARK = <previous>`. If no row changes, another runner got there first and nothing is committed.

If loading fails or counts disagree, the watermark is unchanged. If the commit itself fails, rerunning the same attempt completes it without another `COPY`.

Loading RAW

1. Validate the local batch: a complete manifest, no coverage gaps, and a JSONL row count equal to the manifest.
2. Refuse an attempt that already has a receipt or RAW rows; a retry needs a new extraction attempt ID.
3. PUT the file to an attempt-specific stage path, so the staged file name and row number form a stable source key.
4. COPY INTO RAW_EVENT_RECORDS, mapping each line's metadata and the full feature into a PAYLOAD VARIANT.
5. Require the COPY count and the attempt-filtered RAW count to equal the manifest.
6. Append a BATCH_ATTEMPT receipt with requested bounds, parameters, window audit, gaps, counts, statuses, copy results, and the full manifest. A failure appends a failed receipt when the connection allows.

Bulk history load labels each candidate `loaded` (one complete, matching receipt), `ready` (no receipt and no RAW rows), or `investigate` (anything else). It loads at most 50 `ready` attempts in window order, rechecks each just before PUT and COPY, and stops on any `investigate` state.

Processing procedure

`PROCESS_LOADED_ATTEMPT(attempt_id)` processes exactly one loaded attempt using Snowflake's injected session. Snowflake procedures are not automatically atomic, so the procedure manages its own transaction.

```
read receipt (must be complete, gap-free, counts equal) -- no transaction yet
read RAW rows for the attempt; compare count with receipt
project every row with the typed parser (rejects kept with reason)
BEGIN
  MERGE staging              (<= 500 rows per bound JSON parameter, FLATTEN)
  build alias map            (from all accepted staging observations)
  rekey old aliases          (collision checks before any DML)
  MERGE revision fact        (deduplicated by revision key first)
  MERGE dimensions + bridge (3 sites per revision)
  MERGE batch fact; INSERT success process record
COMMIT
on error: ROLLBACK, then INSERT failed process record outside the transaction
```

An uncertain COMMIT response stops for manual inspection instead of guessing success or failure. The procedure returns a JSON summary only after a committed success; errors stay visible to the caller after the failure is recorded.

Typed parser contract

Typed field	GeoJSON source	Rule
source_event_id	id	Required non-empty string, preserved exactly
origin_time	properties.time	Required epoch milliseconds, converted to UTC
source_updated_at	properties.updated	Required epoch milliseconds, converted to UTC
source_status	properties.status	Required; <code>deleted</code> stays a tombstone candidate
associated_ids	properties.ids	Comma-delimited list kept for alias resolution; may be absent
magnitude, magnitude_type, place	mag, magType, place	Nullable; a malformed non-null value is rejected, never cast
longitude, latitude, depth_km	geometry.coordinates	Valid Point required for active records; a deleted record may have no geometry

Invalid rows are kept in staging with a reject reason and their RAW lineage (for example `invalid_source_event_id`, `invalid_geometry`, `invalid_magnitude`). Parser version 2 adds `source_stub_record` for USGS placeholder features with no time, magnitude, or status, and `placeholder_location` for an active record at `[0, 0]`. Unknown and newly added source fields stay in the RAW `VARIANT` and never change the typed contract.

Data model

A small star schema around one fact grain: a distinct canonical event revision. Grains are enforced by `MERGE` keys and SQL uniqueness checks, because Snowflake does not enforce key constraints on standard tables.

Object	Grain (key)	Notes
<code>RAW.BATCH_ATTEMPT</code>	One extract/load attempt (<code>ATTEMPT_ID</code>)	Append-only receipt; <code>LOGICAL_BATCH_ID</code> groups retries
<code>RAW.RAW_EVENT_RECORDS</code>	One staged file row (file name, row number)	Full feature in <code>PAYLOAD VARIANT</code> ; not deduplicated
<code>RAW.UPDATE_WATERMARK</code>	One sweep (<code>SWEEP_NAME</code>)	Committed and previous watermark, sweep attempt, catalog bounds

Object	Grain (key)	Notes
STG_EVENT_REVISION	One RAW observation (file name, row number)	Typed projection, both parser versions, REJECT_REASON
FACT_EVENT_REVISION	(CANONICAL_EVENT_ID, SOURCE_UPDATED_AT, PAYLOAD_HASH)	Source, fetch, and curated clocks; typed attributes; lineage
BRIDGE_EVENT_SITE	Revision key + SITE_KEY	Epicentral distance and within-radius flag; null for a tombstone without coordinates
DIM_DATE, DIM_SITE, DIM_MAGNITUDE_TYPE, DIM_EVENT_STATUS	One date, site, type, or status	Joined on natural values; DIM_DATE is role-played by origin and update dates
FACT_BATCH_RUN	One processed attempt	Returned, written, loaded, processed, and rejected counts; fetch-to-curated seconds
BATCH_PROCESS_ATTEMPT	One procedure call (PROCESS_ATTEMPT_ID)	Append-only; status, counts, failure type and message
EVENT_CURRENT (view)	One canonical event	Latest revision unless it is a tombstone
Health views	One attempt, window, or reject	V_BATCH_HEALTH, V_RECEIPT_WINDOW_AUDIT, V_EVENT_REJECTS

Current-state view

```

ROW_NUMBER() OVER (
  PARTITION BY CANONICAL_EVENT_ID
  ORDER BY SOURCE_UPDATED_AT DESC, FETCHED_AT DESC, PAYLOAD_HASH DESC,
           STAGE_FILE_NAME DESC, STAGE_FILE_ROW_NUMBER DESC) AS REVISION_RANK
...
WHERE REVISION_RANK = 1 AND SOURCE_STATUS <> 'deleted'

```

Ranking happens before the deleted filter. Filtering first would let an older active revision reappear after a deletion.

Batch health: V_BATCH_HEALTH joins receipts, RAW, staging, process attempts, and the batch fact per attempt and labels each one, for example LOAD_INCOMPLETE, PENDING_PROCESS, or PROCESS_FAILED. PROCESSED_ROWS includes rejects, so a complete attempt has PROCESSED_ROWS = LOADED_ROWS.

Correctness rules

Rule	Design
Revision identity	<code>(canonical_event_id, source_updated_at, payload_hash)</code> . Candidates are deduplicated before every <code>MERGE</code> , keeping the latest fetch and breaking ties by staged file-row key, so the result does not depend on input order. Different payloads at one update time stay distinct; conflicting content under one source key fails
Alias resolution	Each preferred ID is linked to its associated IDs, transitively. The canonical ID is the lexicographically first ID in each connected component, so a preferred-ID change keeps one identity. The map is rebuilt from all accepted staging rows after the staging <code>MERGE</code>
Alias rekey	When a new alias changes an old canonical key, the latest-fetched row wins collisions, its three bridge rows are checked, and other copies are removed before surviving keys are updated, all inside the batch transaction. Every delete and update must match its preflight row count
Tombstones	A deletion is the latest revision with <code>deleted</code> status. Absence from a response is never a deletion, and a stale replay cannot outrank the tombstone
Old events	Origin time never decides the current revision, so an old event updated recently is still captured
Distance	Haversine great-circle distance with a 6,371.0088 km mean Earth radius from each public site center; a distance equal to the radius counts as within. Epicentral only; depth is not used
Stub records	Placeholder IDs are not fed into alias resolution, because the full records already carry the same links

Interfaces

Commands

`make help` lists the targets. Targets that reach USGS or Snowflake preview unless `EXECUTE=1` is set.

Target	What it does
<code>setup, lint, typecheck, test, coverage, check</code>	Locked install, ruff, mypy, the offline suite, and coverage with an 80% floor; <code>check</code> is what CI runs

Target	What it does
<code>extract, plan-history, capture-history</code>	Fetch one batch, print the 180 history windows, or capture missing history windows
<code>bootstrap</code>	Create the project tables and the procedure
<code>load-preview, load, load-history</code>	Check or load one saved batch, or bulk-load ready history windows
<code>process</code>	Call the procedure for up to <code>MAX</code> loaded attempts, checking counts before and health after each
<code>quality, uniqueness, postrun, metrics, analysis</code>	Read-only reconciliation, duplicate-key, post-run, latency, and analysis queries
<code>integration</code>	Compile all SQL live with <code>EXPLAIN</code> and run the read-only checks

Configuration

Setting (<code>settings.py</code>)	Value
Sites	Seattle (47.6062, -122.3321), San Francisco (37.7749, -122.4194), Anchorage (61.2181, -149.9003); 250 km radius each
History horizon	5 years of origin time
Result limits	20,000 USGS maximum; 10,000 target per leaf request
Timeouts	30 s per request; 180 s per source window
Bulk load cap	50 history windows per run

Credentials

Commands read the `quakewatch_project` profile from `~/.snowflake/config.toml`, require key-pair authentication and `QUAKEWATCH_ROLE`, and prompt for the key passphrase in the terminal. They never take a secret from the repository or a command-line argument. In CI the same commands read four `QUAKEWATCH_SNOWFLAKE_*` repository secrets.

Failure handling

Failure	Behaviour	Recovery
Source timeout or 5xx	Bounded retries, then split, within the window budget	Window becomes a recorded gap; rerun later or with smaller slices
Counts disagree	Retry, then split; no partial file is written	Same as above
Process killed mid-capture	Attempt marked <code>failed</code> with zero rows	Resume from saved child checkpoints
<code>COPY</code> count mismatch	No success receipt; failed receipt when possible	New extraction attempt
Transform error	Model writes roll back; failed process record appended	Call the procedure again from RAW, no refetch
Uncertain <code>COMMIT</code>	Stops without claiming an outcome	Inspect state, then retry
Alias collision anomaly	Stops before any DML	Investigate the duplicate or orphaned keys
Partial update sweep	Watermark not advanced	Retry the sweep; the overlap recovers changes
Concurrent sweep commit	Compare-and-set lets only one runner advance	The other run reports no change
Bad data merged	Time Travel and zero-copy clones on the affected table	Query or clone <code>BEFORE</code> the statement; rebuild from RAW

Security and privacy

- Key-pair authentication only, with the key restricted to the project role; the admin role is used only for one-time bootstrap and read-only account-usage metering.
- Credentials live in each user's Snowflake config; the repository holds only `snowflake-config.example.toml`.
- GitHub push protection, CodeQL, and Dependabot are enabled; commits are signed; a full-history scan found no committed secrets.
- Only public city-center coordinates are used. Local raw data under `data/` is ignored by Git.
- USGS data is credited to the U.S. Geological Survey and linked to official event records.

Testing and verification

Layer	How	When
Offline tests	340 tests on deterministic synthetic fixtures; Snowpark writers run against fake sessions; 86% coverage of the package	Every push and pull request
Static checks	ruff lint and format, mypy	Every push and pull request
Live integration	<code>EXPLAIN USING TEXT</code> on every SQL statement the procedure issues and every reviewed query, plus the procedure-exists, post-run, and uniqueness checks	Manual GitHub workflow or <code>make integration</code>
Recorded drills	Isolated fixture database: update, tombstone, stale replay, old-origin update, failed-transform retry, clone and Time Travel	Once, with evidence

`EXPLAIN` compiles against the live schema without running anything, so schema drift fails cheaply. Runtime behaviour such as `MERGE` results, transactions, and rollback is covered by the drills, not by CI.

Cortex brief

A reviewed SQL query produces bounded aggregates per site and window (event count, nearest event, distance, magnitude, status, record age). `AI_COMPLETE` with `claude-haiku-4-5` turns one aggregate into a sentence. The output is accepted only if a local screen passes:

- Every fact appears exactly as in the SQL row: site, count, 250 km radius, event ID, distance from the site, magnitude, status, record age, and the word "modeled".
- No hazard or action language (risk, shaking, damage, safe, should, alert, and similar) and no complete-coverage claim.
- Distance is not anchored to the event ID, and the sentence is complete.

Rejected or failed briefs fall back to the SQL facts. Results are cached by model and case, and accepted text still gets a human comparison with the SQL.

Design decisions

Each decision is recorded as an [architecture decision record](#) in the repository.

ADR	Decision	Alternative not taken	Trade-off accepted
0001	Bounded, manually invoked batches	Snowpipe Streaming or Kafka	No sub-minute freshness
0002	Append-only revisions keyed by event, update time, and payload hash	Overwrite one row per event	Fact table grows; current state needs a ranking view
0003	Deletions kept as tombstone revisions	Delete rows, or infer deletion from absence	View must rank before filtering
0004	Watermark advances only after full reconciliation, by compare-and-set in Snowflake	Advance per window, or keep it in a local file	An incomplete sweep blocks progress until fixed
0005	One writer per table; one transaction per batch	Several writers, or per-table commits	Depends on the account's Snowpark runtime
0006	Python commands as the only orchestrator	Airflow or Snowflake Tasks	No schedule or alerting yet
0007	No clustering or special partitioning	Clustering keys or search optimisation	Tuning at scale is not demonstrated
0008	Separate reject reasons for USGS stub records	One generic invalid-time reason	Needed a one-off data migration
0009	Credentials in per-user Snowflake config	.env file	Manual key-pair setup
0010	Live check compiles SQL with <code>EXPLAIN</code>	Run the procedure in CI	Runtime behaviour proven only by drills

Known limitations and open work

- The catalog-wide sweep has not completed live: both USGS endpoints time out for one minute on 9 Oct 2023, so no watermark is committed (issue #4).
- Three history windows are USGS source gaps (issue #5).
- There is no scheduler or alerting; latency was measured only on a manually curated backfill (issue #6).

- The procedure is not run end to end in CI; a disposable zero-copy clone test is planned (issue #7).
- The deployed procedure bundle predates a code reformat and differs byte-for-byte, with identical behaviour (issue #8).
- Failure and unfinished-attempt projections in `FACT_BATCH_RUN`, and counts of rows removed by alias rekeys, are not yet recorded.

References

- Repository: [README](#), [data dictionary](#), [operations reference](#), [architecture decisions](#), [results](#), [SQL](#)
- USGS: [FDSN Event API](#), [ComCat](#), [credit policy](#)
- Snowflake: [COPY INTO table](#), [Python procedures](#), [transactions](#), [MERGE](#), [FLATTEN](#), [Time Travel](#), [cloning](#), [AI_COMPLETE](#)